

LEGAL EASE AI-POWERED LEGAL DOCUMENT GENERATOR

Legal Ease -Deliver Intelligent Legal Document And Insights

Definition:

LegalEase is an AI-powered legal document generation platform designed to simplify the creation of commonly required legal documents. The system collects relevant information from users through structured forms, processes the input using artificial intelligence, and generates clear, organized legal document drafts. The platform is designed with secure user authentication, document storage, validation, and controlled AI processing to provide a convenient document-generation workflow.

Team Members Name & Roles:

- 1. Name: Yashmika S (Team Leader) Role: Voice over
- 2. Name: Sri lakshna S (Team Member) Role: Documentation
- 3. Name: Sharmila J M (Team Member) Role: APT Video
- 4. Name: Yoga dharshni B (Team Member) Role: Coding
- 5. Name: Sowmiya S (Team Member) Role: Demo Video

1.0 Executive Summary & Platform Purpose

LegalEase is a scalable AI-powered application developed to reduce the complexity involved in preparing structured legal document drafts. Traditional document preparation often requires users to understand legal formats, identify required clauses, and manually arrange information. LegalEase addresses this difficulty by combining structured data collection with AI-assisted document generation.

1.1 Core Objectives

- Automated Document Generation: Generate structured legal document drafts from user-provided information.
- AI Assistance: Use an AI model to organize user inputs into an appropriate legal-document format.
- User-Friendly Workflow: Provide simple forms and guided fields so users can enter required information easily.
- Secure Data Handling: Protect user accounts and document information using authentication and secure storage practices.
- Document Management: Allow users to create, view, save, and manage generated documents.
- Validation & Reliability: Validate required fields and handle invalid inputs or service failures safely.

1.2 High-Level Platform Capabilities

- 1. User Authentication: Secure registration and login using token-based authentication and protected credentials.
- 2. Legal Template Management: Maintains structured document templates and required fields for different document types.
- 3. AI Document Generation: Sends validated user information to the AI service and receives a structured document draft.
- 4. Document Storage: Stores generated documents and related metadata for later access.
- 5. Centralized Middleware: Handles authentication, validation, sanitization, and error responses.
- 6. Modular Design: Separates controllers, services, models, AI integration, and API routes.

2.0 Scenario-Based Case Study & Requirements

2.1 Background Story

Alex needs to prepare a legal document for a personal or business requirement but is unfamiliar with formal legal-document structures. Instead of manually searching for formats and drafting every section, Alex uses LegalEase, selects the required document type, enters the necessary information, and receives an AI-assisted structured draft that can be reviewed before use.

2.2 Problem Formulation

Problem Area	LegalEase Function	Purpose / Outcome
Manual legal drafting	Guided document-generation workflow	Reduces the effort of manually arranging legal-document sections.
Complex legal terminology	Structured input forms with required fields	Helps users provide the information needed for the selected document type.
Different document formats	Reusable document templates and document-type selection	Ensures the draft follows the selected document structure.
AI-assisted drafting need	AI service / Gemini API integration	Transforms validated user information into an organized legal-document draft.
Validation and review	Input validation, sanitization, and draft review	Reduces invalid input and supports professional review where required.

2.3 Software & Hardware Environment Requirements

Software Specifications:

- Operating System: Windows 10/11, macOS, or Linux.
- Runtime Environment: Node.js 16.0+ or a suitable current LTS version.

- Package Manager: npm.
- Database: MongoDB.
- Backend Framework: Express.js.
- AI Integration: Google Gemini API or another supported generative AI service.
- Core Dependencies: express, mongoose, jsonwebtoken, bcryptjs, dotenv, and the selected AI SDK.

3.0 System Architecture & Design Patterns

LegalEase follows a multi-layered architecture adapted for an AI-enabled web application. The architecture separates the presentation layer, API routes, business logic, AI service integration, and database operations. This separation improves maintainability, testing, security, and future expansion.

3.1 Architecture Blueprint

Architecture Layer / Component	Main Responsibility	Flow / Output
User / Client	Provides the user-facing entry point.	User request → Web interface
Web Interface / Form	Collects document type and required user information.	Structured form data → API request
Authentication & Validation Middleware	Checks access, validates required fields, and sanitizes input.	Validated request → API routes
API Routes & Controllers	Receives requests and invokes application services.	Request → Controller → Service
Legal Document Service	Coordinates templates, user inputs, AI processing, and document generation.	Structured generation workflow
Template Engine	Provides reusable document structure and required sections.	Template + input data → draft structure
AI Service / Gemini API	Assists with organizing validated information into a draft.	Prompt/input → AI-generated content
Validation & Formatting	Checks generated output and prepares a consistent document format.	Validated draft → persistence
MongoDB Database	Stores users, templates, generated documents, and related metadata.	Stored document → response

3.2 Architectural Flow

1. User registers or logs into the LegalEase platform.
2. Authentication middleware verifies the user's access token.
3. User selects a legal document type.
4. The system displays the required information fields for the selected document.

5. User submits the completed form.
6. Controller validates the request and passes the data to the document-generation service.
7. The service prepares a structured prompt or template and communicates with the AI service.
8. The generated legal document draft is validated and formatted.
9. The document and metadata can be stored in MongoDB.
10. A formatted response is returned to the client with an appropriate HTTP status code.

4.0 Data Architecture & ER Modeling

The LegalEase database uses a document-oriented MongoDB structure. Mongoose models define relationships and validation rules for users, document templates, generated documents, and related metadata.

4.1 Entity Relationship Diagram Structure

Primary Collections & Relationships:

- **USERS Collection:** Stores user identity, authentication information, roles, and account timestamps.
- **DOCUMENT_TEMPLATES Collection:** Stores supported document types, fields, sections, and template metadata.
- **GENERATED_DOCUMENTS Collection:** Stores generated document content, selected document type, owner, and timestamps.
- **DOCUMENT_VERSIONS Collection (Optional):** Stores previous versions of generated documents for revision history.
- **AI_REQUESTS Collection (Optional):** Stores generation metadata such as request status, timestamps, and document references.

4.2 Entity Mapping Matrix

User → Generated Documents: One-to-Many. A registered user can create multiple legal document drafts.

Document Template → Generated Documents: One-to-Many. One document template can be used to generate multiple document drafts.

Generated Document → Document Versions: One-to-Many when version tracking is enabled.

Entity	Related Entity	Relationship	Description
USERS	GENERATED_DOCUMENTS	One-to-Many (1:N)	One registered user can create multiple legal document drafts.
DOCUMENT_TEMPLATES	GENERATED_DOCUMENTS	One-to-Many (1:N)	One document template can be used to generate multiple document drafts.
GENERATED_DOCUMENTS	DOCUMENT_VERSIONS	One-to-Many (1:N)	One generated document can have multiple versions when version tracking is enabled.
GENERATED_DOCUMENTS	AI_REQUESTS	One-to-Many (1:N)	A generated document may be associated with AI generation request records when tracking is enabled.

5.0 Executive Summary & System Vision

LegalEase is designed as an AI-assisted legal document generation system that transforms structured user information into organized legal-document drafts. The system combines guided input forms, reusable document structures, AI processing, secure storage, and document management to provide a simple end-to-end workflow.

5.1 Key System Attributes

- **AI-Assisted Generation:** Converts structured user information into a readable legal-document draft.
- **Secure Authentication:** Uses protected authentication and password hashing.
- **User Personalization:** Maintains documents associated with individual user accounts.
- **Validation:** Checks required information before document generation.
- **Error Handling:** Provides controlled responses when invalid input or AI service errors occur.
- **Scalable Architecture:** Separates AI, document, database, and authentication services for future expansion.

6.0 End-to-End System Architecture Blueprint

Step	System Component	Process	Result / Output
1	User	Accesses LegalEase and starts the workflow.	User session initiated
2	Web / Client Interface	Displays document options and guided forms.	Selected document type

3	Login / Authentication	Verifies credentials and access token.	Authenticated user
4	Document Type Selection	User selects the required legal document category.	Template category selected
5	Guided Input Form	Collects the required information for the selected template.	Structured user input
6	API Request	Sends the completed form to the backend.	Generation request
7	Authentication + Validation + Sanitization	Checks access and validates/sanitizes submitted data.	Safe validated request
8	Controller	Routes the request to the document-generation service.	Service invocation
9	Legal Document Service	Coordinates template selection and AI processing.	Generation workflow
10	Template Engine	Applies reusable legal-document structure.	Template-based structure
11	Generated Draft Validation & Formatting	Checks the draft and prepares the output format.	Validated formatted draft
12	MongoDB Persistence	Stores generated content and metadata where persistence is enabled.	Saved document
13	Saved / Versioned Document	Maintains the current draft and optional versions.	Retrievable document
14	Response to Client	Returns the generated result to the interface.	Document displayed to user
15	User Reviews / Edits / Saves / Exports	User reviews and manages the draft where supported.	Final user-managed draft

6.1 Data Modeling & User Domain Architecture

The core data structure centers on Users, Document Templates, and Generated Documents. These entities allow the system to associate generated legal-document drafts with the user who created them and the document type selected.

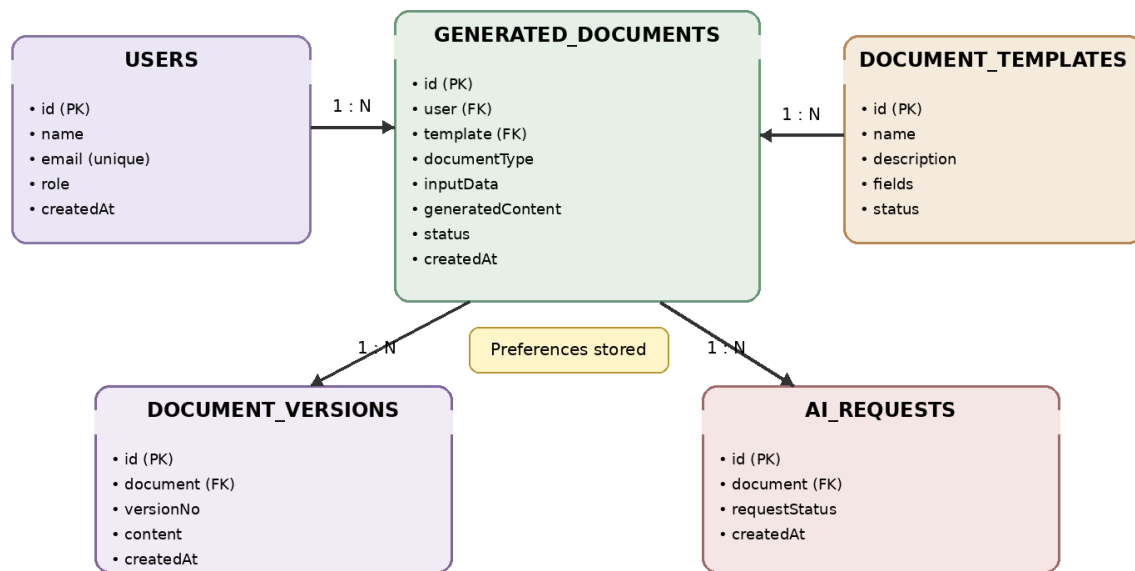
6.2 User Account Profile

- Identity Constraints: Requires a unique email address or username.
- Credential Protection: Passwords are stored using secure cryptographic hashing rather than plain text.
- Account Metadata: Stores account creation and update timestamps.
- Access Control: Protected routes require valid authentication.

6.3 Legal Document Preferences

Preference / Data Element	Stored / Related Information	Purpose
Document Type	Selected legal document category and template reference	Identifies the structure to be used for generation.
User Inputs	Structured information supplied through the guided form	Provides the data required to prepare the document.
Generated Content	AI-assisted draft content linked to the generated document	Stores the resulting draft when persistence is enabled.
Status	Draft, Generated, Edited, or Archived	Tracks the current state of the document.
Version History	Previous document versions when enabled	Supports revision tracking and recovery of earlier drafts.
AI Request Metadata	Generation status, timestamps, and document reference when enabled	Provides traceability for AI-assisted generation.

Legal Document Preferences - ER / Data Flow View



PK = Primary Key FK = Foreign Key 1:N = One-to-Many relationship

The diagram shows how user preferences and document-generation data are associated with generated documents. A user can create multiple generated documents, a document template can support multiple generated documents, and optional version and AI-request records can be associated with each generated document.

1. User Entity:

- id (Primary Key - ObjectId)
- name (String, Required)
- email (String, Required, Unique)
- password (String, Hashed, Required)

- role (String, Default: 'user')
- createdAt (Date)

2. Document Template Entity:

- id (Primary Key - ObjectId)
- name (String, Required)
- description (String)
- fields (Array)
- status (String)
- createdAt (Date)

3. Generated Document Entity:

- id (Primary Key - ObjectId)
- user (Foreign Key referencing User Collection)
- template (Foreign Key referencing Document Template Collection)
- inputData (Object)
- generatedContent (String)
- status (String)
- createdAt (Date)

Key Structural Mappings:

- User to Generated Document (One-to-Many): A user can generate multiple documents.
- Template to Generated Document (One-to-Many): A template can be used for multiple generated documents.
- CRUD Document Management: Users can create, view, update, and manage their saved drafts.
- Validation Safeguards: Required fields and input formats are validated before generation.
- Sanitization Safeguards: User input is sanitized to reduce injection and unsafe payload risks.

Roles and Responsibilities

Admin:

- Responsible for managing users and document templates.
- Monitors system health and application logs.
- Reviews and maintains document-template configuration.

Registered User:

- Creates and maintains a personal collection of generated documents.
- Selects a legal document type and provides required information.
- Requests AI-assisted document generation.
- Reviews, edits, saves, or exports generated drafts where supported.

Core System Script Implementations:

Application Entry Point (index.js):

The index.js file serves as the main entry point of the LegalEase backend. It initializes the Express server, loads environment variables, configures middleware, establishes the MongoDB connection, registers API routes, and starts the server after the database connection is successfully established.

Epic 4: Database Configuration

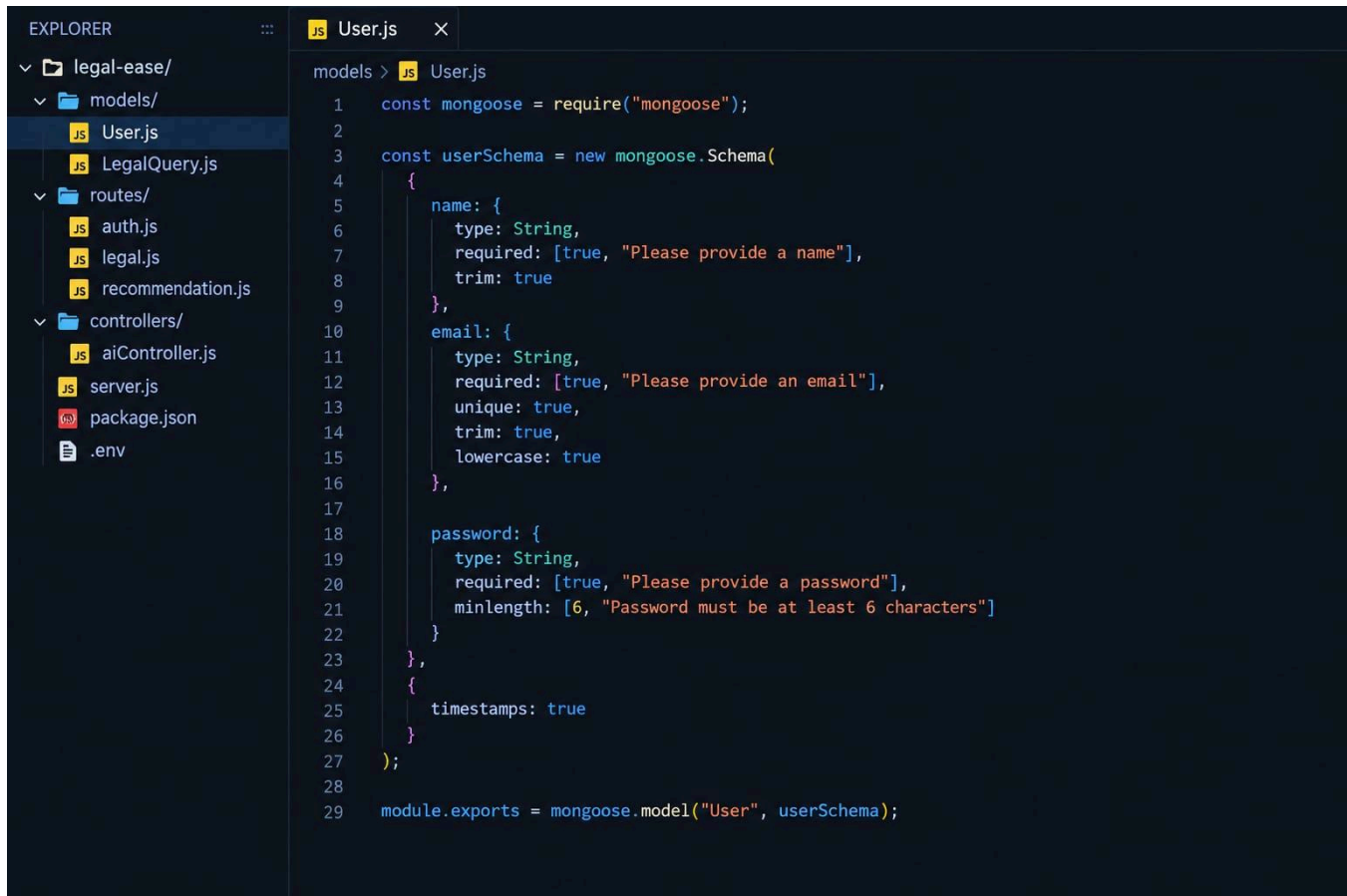
API Testing Demo:

- Configure MongoDB.
- Install Mongoose using npm.
- Create the database connection.
- Create User, Document Template, and Generated Document schemas.
- Connect the application to MongoDB before processing document requests.

Create Database Connection:

The database connection module establishes communication between the LegalEase backend and MongoDB using Mongoose. The application should verify the database connection before performing operations such as user registration, document creation, or document retrieval.

Mongoose Schema Definitions



The image shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project structure for 'legal-ease/' with folders 'models/' and 'routes/' and files 'User.js', 'LegalQuery.js', 'auth.js', 'legal.js', 'recommendation.js', 'aiController.js', 'server.js', 'package.json', and '.env'. The code editor shows the content of 'User.js' in the 'models' folder. The code defines a Mongoose schema for a 'User' model with fields 'name', 'email', and 'password', each with specific validation rules. The schema is then used to create a Mongoose model.

```
1  const mongoose = require("mongoose");
2
3  const userSchema = new mongoose.Schema(
4    {
5      name: {
6        type: String,
7        required: [true, "Please provide a name"],
8        trim: true
9      },
10     email: {
11       type: String,
12       required: [true, "Please provide an email"],
13       unique: true,
14       trim: true,
15       lowercase: true
16     },
17
18     password: {
19       type: String,
20       required: [true, "Please provide a password"],
21       minlength: [6, "Password must be at least 6 characters"]
22     }
23   },
24   {
25     timestamps: true
26   }
27 );
28
29 module.exports = mongoose.model("User", userSchema);
```

User Schema:

```
.js legal.js x
routes > .js legal.js
1  const express = require('express');
2  const LegalQuery = require('../models/LegalQuery');
3  const { analyzeLegalQuestion } = require('../controllers/aiController'); // AI module
4  const router = express.Router();
5
6  // POST /api/legal/ask - Ask a legal question
7  router.post('/ask', async (req, res) => {
8    try {
9      const { userId, question, category } = req.body;
10     if (!userId || !question) {
11       return res.status(400).json({
12         success: false,
13         message: 'User ID and question are required'
14       });
15     }
16
17     // Get AI response and recommendations
18     const aiResult = await analyzeLegalQuestion(question, category);
19     const { summary, category: detectedCategory, recommendations } = aiResult;
20
21     // Save to database
22     const query = await LegalQuery.create({
23       userId,
24       question,
25       category: detectedCategory || category,
26       aiResponse: summary,
27       recommendations,
28     });
29
30     res.status(201).json({
31       success: true,
32       message: 'Legal query processed successfully',
33       data: {
34         id: query._id,
35         question: query.question,
36         category: query.category,
37         aiResponse: query.aiResponse,
38         recommendations: query.recommendations,
39         createdAt: query.createdAt
40       }
41     });
42   } catch (error) {
43     console.error('Legal query error:', error);
44     res.status(500).json({
45       success: false,
46       message: error.message || 'Something went wrong'
47     });
48   }
49 });
50
51 module.exports = router;
```

The User schema defines account information, authentication credentials, role information, and timestamps.

Document Template Schema:

```
JS User.js x
models > JS User.js
1  const mongoose = require('mongoose');
2
3  const userSchema = new mongoose.Schema(
4    {
5      name: {
6        type: String,
7        required: [true, "Please provide a name"],
8        trim: true
9      },
10     email: {
11       type: String,
12       required: [true, "Please provide an email"],
13       unique: true,
14       trim: true,
15       lowercase: true
16     },
17     password: {
18       type: String,
19       required: [true, "Please provide a password"],
20       minlength: [6, "Password must be at least 6 characters"]
21     }
22   },
23   {
24     timestamps: true
25   }
26 );
27 module.exports = mongoose.model("User", userSchema);
```

The Document Template schema defines the supported legal document types, required fields, sections, and template metadata.

Generated Document Schema:

The Generated Document schema stores the user reference, selected document type, submitted input data, generated draft content, status, and timestamps.

DB Connection:

This module establishes a connection between the LegalEase application and MongoDB using Mongoose. It ensures that the database is available before backend operations are processed.

Project Executions:

The LegalEase backend can be executed locally using Node.js. Before running the application, ensure that MongoDB is running and the required environment variables, including the AI service API key, are configured correctly. The REST APIs can be tested using Postman or Thunder Client.

Running the Backend Server:

Install all required dependencies:

```
npm install
```

Configure the environment variables in the .env file.

Start the development server:

```
npm run dev
```

After successful execution, the server connects to MongoDB and starts listening for incoming API requests.

API Testing:

Register API:

POST /api/auth/register – Creates a new LegalEase user account.

Login API:

POST /api/auth/login – Authenticates the user and returns an access token.

Document Types API:

GET /api/documents/templates – Retrieves available legal document types or templates.

Generate Document API:

POST /api/documents/generate – Accepts validated user information and generates an AI-assisted legal document draft.

Saved Documents API:

GET /api/documents – Retrieves documents associated with the authenticated user.

Recommendation / Assistance API:

POST /api/ai/assist – Sends a supported request to the AI service for document-generation assistance or structured legal content suggestions.

7.0 Security Considerations

- Passwords are stored using secure hashing.
- Authentication tokens are validated on protected routes.

- Input validation is performed before processing.
- Request sanitization helps reduce injection risks.
- Sensitive API keys are stored in environment variables rather than source code.
- Access to user documents is restricted to authenticated and authorized users.

8.0 Testing Strategy

- Unit Testing: Tests individual services, validators, and utility functions.
- API Testing: Tests registration, login, template retrieval, generation, and document-management endpoints.
- Integration Testing: Verifies communication between the backend, MongoDB, and AI service.
- Validation Testing: Checks missing, invalid, or incorrectly formatted user inputs.
- Security Testing: Verifies authentication, authorization, sanitization, and protected routes.
- User Acceptance Testing: Confirms that users can complete the document-generation workflow successfully.

9.0 Expected Output

After successful processing, LegalEase produces a structured legal-document draft based on the selected document type and user-provided information. The generated document can be reviewed and, depending on the implemented features, edited, saved, or exported for further use.

10.0 Advantages

- Reduces the time required for preparing structured document drafts.
- Provides a guided workflow for users unfamiliar with legal document formats.
- Supports reusable document types and templates.
- Provides AI-assisted organization of supplied information.
- Enables centralized storage and management of generated drafts.
- Provides a modular architecture that can support additional document types.

11.0 Limitations

- AI-generated content may require human review for accuracy and completeness.
- Legal requirements can differ by jurisdiction and document type.
- The system should not be treated as a substitute for professional legal advice.
- AI service availability and response quality may affect generation.
- Some specialized legal documents may require expert